



React Router

In the old days...

Traditioneel wordt er vanuit de browser een request gemaakt per pagina, deze gaat dan alle nodige JS en CSS downloaden en de HTML renderen die teruggestuurd wordt van de server.

Met de komst van SPA's spreken we van client side routing

Er is maar 1 HTML pagina dus er moet maar 1 request uitgevoerd worden naar de server.

Routing gaat nu dus voorzien worden in de browser (of client).



Hoe werkt het?

- URL updaten zonder een nieuwe request uit te voeren
- Browser API's → History API
- Binnen JavaScript de routes afhandelen en UI updaten

Installatie

Refresher, React is geen framework maar een **library**

React voorziet niets voor routing out-of-the-box. Met alles dat React aanbiedt in hun library kunnen we in principe onze eigen routing voorzien.

Natuurlijk hoeven we deze zelf niet te schrijven. Er bestaat een community standard package voor client side routing nl. [React Router](#).

React Router v7 documentatie

<https://reactrouter.com/>



```
npm i react-router
```

Routing modes

Wat zijn router modes?

React Router v7 breidt het concept van routing uit naar **drie afzonderlijke modes** die elk passen bij een andere mate van complexiteit en behoeften van een applicatie. Deze modes zijn ontworpen als **"opt-in progressie"**: je start eenvoudig, en als je app groeit, kun je upgraden naar krachtigere patterns zonder je hele router te herschrijven.

Declarative mode

Voor Wie?

Voor beginners of kleine apps. Je definieert routes direct in je JSX.

Kenmerken:

- Gebruik van `<Routes>` en `<Route>` componenten
- Route-informatie in componentstructuur
- Geen loaders of actions

Data mode

Voor Wie?

Voor apps die nood hebben aan meer structuur en complexiteit

Kenmerken:

- Gebruikt `createBrowserRouter()` i.p.v. `<Routes>`
- Ondersteunt `loader`, `action`, `errorElement`, enz.
- Data fetching gebeurt buiten de component, vóór het renderen

Framework mode

Voor Wie?

Voor grotere apps die een volledig framework nodig hebben.
Gebouwd op Vite, Remix is sinds v7 opgegaan in React Router zelf

Kenmerken:

- Routing wordt automatisch gegenereerd op basis van een bestandsstructuur
- Volledige integratie met data loading en form actions
- Server side rendering

The background is a deep purple space scene featuring a large, intricate nebula with wispy, glowing structures. Overlaid on this are several thin, light-purple geometric lines: a grid of squares, a large circle centered in the upper half, and several smaller circles and intersecting lines that create a technical or architectural feel.

Routers

Routers

Een Router houdt bij waar de gebruiker in onze app is (de huidige URL) en hoe hij daar gekomen is (URL-geschiedenis). Ook biedt hij de mogelijkheid om naar andere pagina's te navigeren.

Waar die URL-info wordt opgeslagen, hangt af van je omgeving - browser, test, of server. Daarom biedt React Router verschillende router-types.

BrowserRouter

- Voor gebruik in de browser
- Gaat de URL in de adresbalk updaten
- Gebruikt de browser zijn native History API om te navigeren

HashRouter

- Gebruikt het hash gedeelte van de url
- Ondersteuning voor oudere browsers
- Hosting die geen client side routing ondersteunen
- Wordt afgeraden om deze nog te gebruiken in moderne omgevingen

MemoryRouter

- Houdt url history bij in eigen geheugen
- Ideaal voor omgevingen die geen toegang hebben tot externe history data zoals die van de browser
 - bv. binnen een test omgeving

HydratedRouter

- Hergebruikt server-side data en routes bij client-side renderen na SSR — dus geen dubbele API-calls.
- Versnelt de page load door direct te hydrateren met de meegeleverde loaderData.

StaticRouter

- Renderen van routes binnen een node omgeving
- Gebruikt voor server side rendered apps

Router configuratie

Belangrijkste properties:

- path: Het pad in de url
- element: De `component` die we willen zien op het ingestelde pad
- children: Zijn er `geneste routes` die vallen onder het pad

Voorbeeld

Components

Routes renderen

`<RouterProvider />`

- Router configuratie wordt hier doorgegeven via de `router` prop
- Zorgt ervoor dat de router context overheen heel de app beschikbaar is

Routes renderen

`<Outlet />`

- Gebruikt in parent Route
- Gaat geneste Route components renderen

Navigeren

`<Link />`

- Vergelijkbaar met een normale `<a>` tag
- Gaat navigeren naar een andere pagina na klik
- Enkel voor interne links, voor externe kan je gwn een `<a>` tag gebruiken
- Link props
 - `to`: verwijst naar het path van een Route

Navigeren

`<NavLink />`

- Zelfde als Link component maar weet wanneer hij actief is obv de huidige locatie
- Handig voor gebruik als navigatie item in hoofdmenu van app

Voorbeeld

The background is a deep purple space scene featuring a large, intricate nebula with wispy, glowing purple and magenta clouds. Overlaid on this are several thin, light-purple geometric lines: a grid of squares, a large circle centered in the frame, and several smaller circles and intersecting lines that create a technical or architectural feel. The word "Hooks" is written in a clean, white, sans-serif font on the left side.

Hooks

useLocation()

- Geeft het huidige locatie object terug
- Handig om te luisteren naar locatie veranderingen binnen de app

useNavigate()

- Geeft een functie terug om programmatisch te kunnen navigeren naar andere Routes

useParams()

- Geeft een object terug met alle params die in de huidige url staan
- `/users/:id` ⇔ `/users/id-1234`

Advanced: Lazy loading

Refresher, Code-splitting

Voor kleinere apps is het geen probleem om deze volledig in 1 JS bestand te bundelen, maar wat als onze app verder moet groeien en ons JS bestand enkel maar groter wordt?

Hier kunnen we lazy loading toepassen om aparte stukken van onze code enkel beschikbaar te maken als deze door de gebruiker opgevraagd wordt.

lazy

- React Router voorziet een eigen API om routes te kunnen lazy loaden
- <https://reactrouter.com/start/data/custom#3-lazy-loading>

The background is a deep purple space scene featuring a large, intricate nebula with wispy, glowing purple and magenta clouds. Overlaid on this are several thin, light-purple geometric lines: a grid of squares, a large circle on the left, a large circle in the center, and a smaller circle in the upper right. The text 'Vragen?' is written in a clean, white, sans-serif font.

Vragen?

